~\Desktop\Untitled-1.cpp

```cpp
class ThreadPool {
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex mtx;
    std::condition_variable cv;
    bool stop = false;
public:
    // 构造函数：启动固定数量的线程
    ThreadPool(size_t threads) {
        for(size_t i = 0; i < threads; ++i) {
            workers.emplace_back([this] {
                while(true) {
                    std::function<void()> task;
                    {
                        // 锁范围仅限于取任务
                        std::unique_lock<std::mutex> lock(mtx);
                        // 等待直到：停止 或 有任务
                        cv.wait(lock, [this]{ return stop || !tasks.empty(); });
                        if(stop && tasks.empty()) return; // 退出线程
                        task = std::move(tasks.front());
                        tasks.pop();
                    }task(); // 执行任务（无锁状态下执行）
                }
            });
        }
    }
    void enqueue(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            if(stop) throw std::runtime_error("ThreadPool stopped");
            tasks.emplace(task);
        }
        cv.notify_one(); // 唤醒一个线程去干活
    }
    ~ThreadPool() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            stop = true;
        }
        cv.notify_all(); // 唤醒所有线程让它们看到 stop 标志
        for(auto &worker : workers) worker.join();
    }
};
int main() {
    ThreadPool pool(4);
    for(int i = 0; i < 8; ++i) {
        pool.enqueue([i] {std::cout << "Task " << i << " running\n";});
    }
    // pool 析构时会自动 join 等待任务完成
    return 0;
}
```